Diabetes-ML-Project

Roshan Joji

2026-05-11

Task 2.1

Reading in the data, stating dimensions, and converting type into factor labelled "Y" and "N".

```
# Reading in data
diabetes_training <- read.csv("diabetes_training.csv")
diabetes_test <- read.csv("diabetes_test.csv")

# Dimensions
dim(diabetes_training)
```

```
## [1] 250   8
```

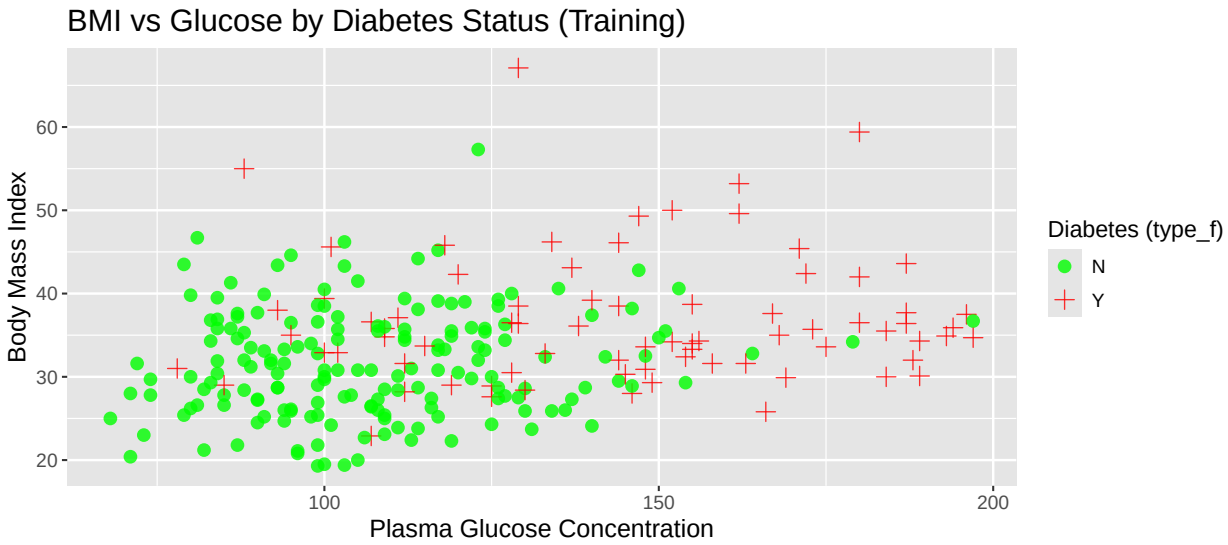
```
dim(diabetes_test)
```

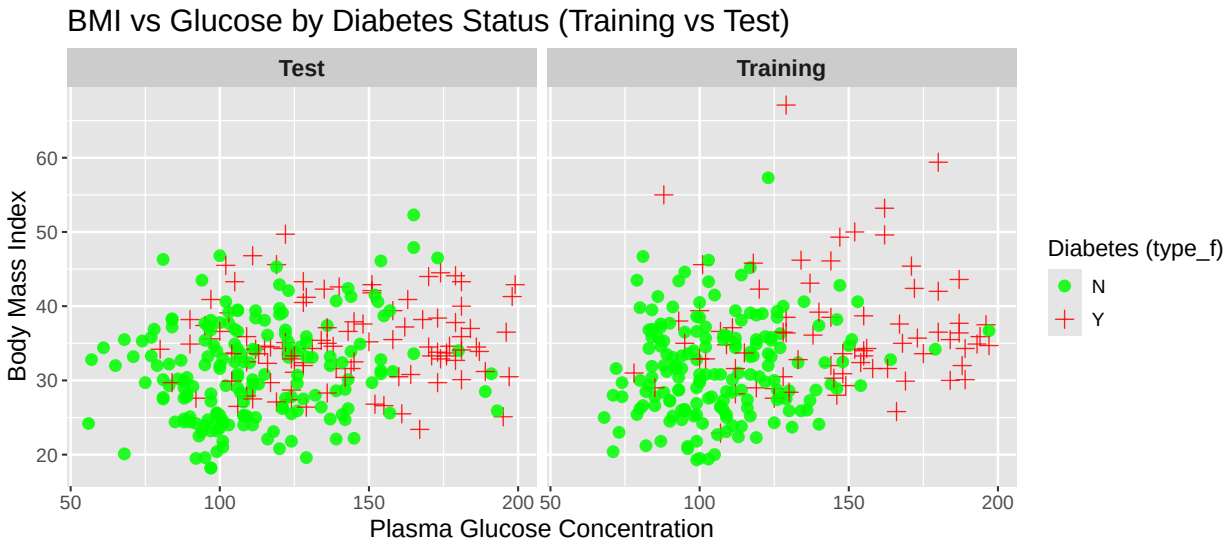
```
## [1] 282   8
```

```
# Convert type to factor
diabetes_training <- diabetes_training %>%
  mutate(type_f = factor(type, levels = c("No", "Yes"), labels = c("N", "Y")))

diabetes_test <- diabetes_test %>%
  mutate(type_f = factor(type, levels = c("No", "Yes"), labels = c("N", "Y")))
```

Task 2.2

First graph shows a scatter plot of bmi against glu, with points labelled by type_f (training data). Second graph shows the same for both training and test data, in a combined faceted scatter plot.





Task 2.3

Finding max. pregnancies in both datasets, and converting "npreg" into a factor "npreg_6_f".

```
# Max number of pregnancies in test and training datasets
max(diabetes_training$npreg)
```

```
## [1] 14
```

```
max(diabetes_test$npreg)
```

```
## [1] 17
```

```
# Creating factor npreg_6_f (training data)
diabetes_training <- diabetes_training %>%
```

```
  mutate(npreg_6_f = fct_collapse(
    as.factor(npreg),
    "6 or more" = as.character(6:20)
  ))

# Creating factor npreg_6_f (test data)
diabetes_test <- diabetes_test %>%
  mutate(npreg_6_f = fct_collapse(
    as.factor(npreg),
    "6 or more" = as.character(6:20)
  ))

# Level ordering
diabetes_training <- diabetes_training %>%
  mutate(npreg_6_f = factor(npreg_6_f,
                            levels = c("0","1","2","3","4","5","6 or more")
                            ))

diabetes_test <- diabetes_test %>%
  mutate(npreg_6_f = factor(npreg_6_f,
                            levels = c("0","1","2","3","4","5","6 or more")
                            ))

# Tabulate results
table(diabetes_training$npreg_6_f)
```

```
##
##         0         1         2         3         4         5 6 or more
##        40        61        32        29        19        11        58
```

```
table(diabetes_test$npreg_6_f)
```

```
##
##         0         1         2         3         4         5 6 or more
##        37        55        47        28        22        20        73
```

Task 2.4

The full binary logistic regression model was fitted before backward elimination.

```
# Fitting full model before backward elimination
model_full <- glm(type_f ~ npreg_6_f +
                    age + I(age^2) + I(age^3) +
                    glu + I(glu^2) + I(glu^3) +
                    bmi + I(bmi^2) + I(bmi^3) +
                    bp + skin + ped +
                    I(glu * bmi),
                  data = diabetes_training,
                  family = binomial)
```

The least significant variable is "npreg_6_f5", with the highest p-value of 0.9075. As the other factors of "npreg_6_f" are not found to be statistically significant either ($p > 0.05$), the whole variable "npreg_6_f" was removed, and this was fitted as a simpler "model_1".

The variable with the highest p-value in "model_1" is "Age^2". However, a lower-order term cannot be removed from the model while a higher order term exists. Therefore, "Age^3" was removed as it was also statistically insignificant, and allowed for evaluation of the lower-order terms.

The least significant variable in the next model, "model_2", is "skin" with a p-value of 0.8543. There are no hierarchical restrictions for this variable, and was therefore removed to create "model_3".

This elimination process was repeated removing statistically insignificant variables "glu*bmi", "glu^3", "glu^2", "bmi^3", "bmi^2", "bp" in order. The final model, "model_9", contains five remaining variables: age, age^2, glu, bmi, and ped, all of which were significant ($p<0.05$).

```
model_9 <- glm(type_f ~
                 age + I(age^2) +
                 glu +
                 bmi +
                 ped,
               data = diabetes_training,
               family = binomial)
```

The final model was evaluated on the test data and used to create probability values. Using a threshold of 0.5 for type 'Y' classification, a confusion matrix was created.

```
# Evaluation of the model on the test data
# Probability prediction
probs <- predict(model_9, diabetes_test, type = "response")

# Convert probabilities to yes or no based on 0.5 threshold
pred <- ifelse(probs > 0.5, "Y", "N")

# Create confusion matrix
cm <- table(Predicted = pred, Actual = diabetes_test$type_f)
cm
```

```
##          Actual
## Predicted   N   Y
##         N 156  47
##         Y  25  54
```

True positive/negative and false positive/negative values were used to calculate model metrics such as error, accuracy, sensitivity, specificity and precision. These metrics will help to compare performance between different models.

```
# Calculate model error, accuracy, sensitivity, specificity, and precision
# Extract values
TN <- cm["N","N"]
FN <- cm["N","Y"]
FP <- cm["Y","N"]
TP <- cm["Y","Y"]

# Error
error <- (FP + FN) / (TP + TN + FP + FN)

# Accuracy
accuracy <- (TP + TN) / (TP + TN + FP + FN)
```

```r
# Sensitivity
sensitivity <- TP / (TP + FN)

# Specificity
specificity <- TN / (TN + FP)

# Precision
precision <- TP / (TP + FP)
```

Task 2.5

As before, the full simple binary logistic regression model was fitted before backward elimination. Backward elimination allowed for the removal of statistically insignificant variables "npreg_6_f", "skin", and "bp" in order. The model can be expressed mathematically as shown below:

$$Y_i \sim \text{Bernoulli}(p_i)$$

$$\log\left(\frac{p_i}{1-p_i}\right) = \beta_0 + \beta_1 age_i + \beta_2 glu_i + \beta_3 bmi_i + \beta_4 ped_i$$

As before, the final model was evaluated on the test data and used to create probability values. A confusion matrix was created using the same 0.5 threshold. The confusion matrix for this model is shown below:

```
##           Actual
## Predicted   N   Y
##         N 160  49
##         Y  21  52
```

Task 2.6

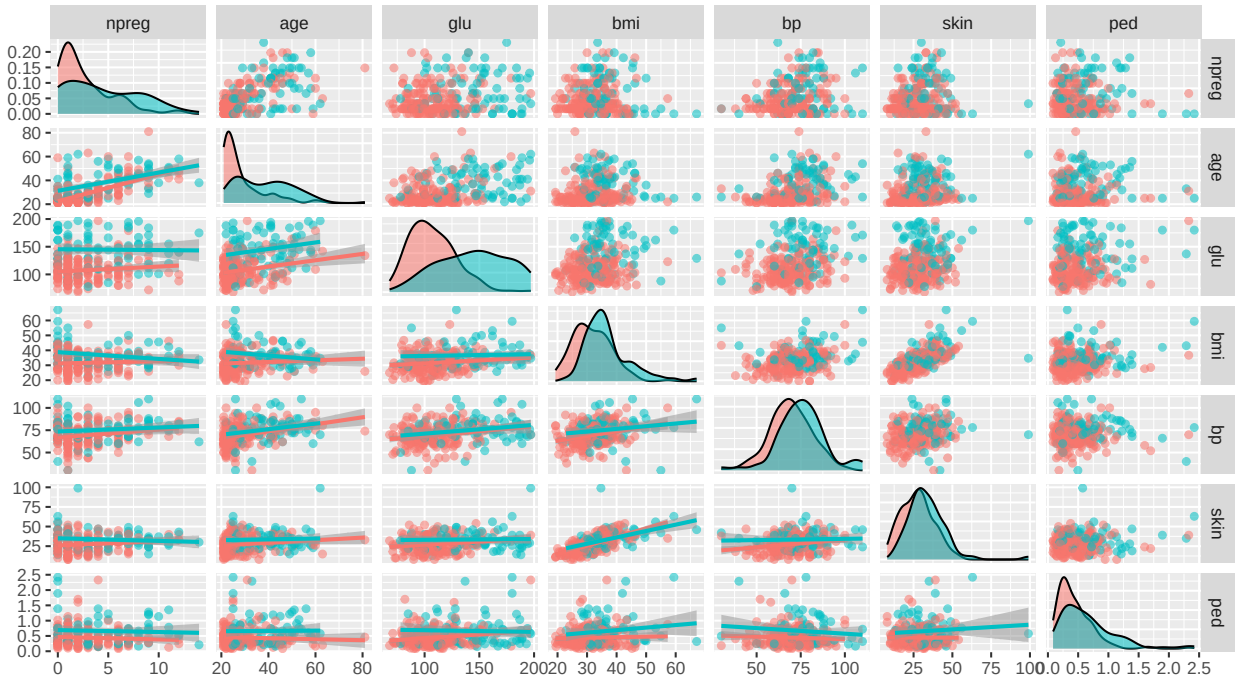
A function was created to compute the F1 score for this simpler model, from the confusion matrix.

```r
# Writing a function to compute 'F1' from confusion matrix produced in 2.5
F1 <- function(cm_simple) {
  TP_simple <- cm_simple["Y","Y"]
  FP_simple <- cm_simple["Y","N"]
  FN_simple <- cm_simple["N","Y"]

  precision_simple <- TP_simple / (TP_simple + FP_simple)
  sensitivity_simple <- TP_simple / (TP_simple + FN_simple)

  f1_score <- 2 * precision_simple * sensitivity_simple / (precision_simple + sensitivity_simple)

  return(f1_score)
}
```

Task 2.7

A matrix of scatter plot for seven covariates, coloured according to "type_f" was created.

```
ggpairs(diabetes_training,
        columns = c("npreg", "age", "glu", "bmi", "bp", "skin", "ped"),
        aes(colour = type_f, alpha = 0.4),
        upper = list(continuous = "points"),
        lower = list(continuous = "smooth"),
        diag = list(continuous = "densityDiag"))
```



The plot shows that "glu" is the strongest discriminator between classes, with higher values being associated with diabetes. A weaker positive trend can be seen with variables "bmi" and "age", while other variables show much overlap, suggesting weak predictive power.

Task 2.8

K-nearest neighbours was applied to the data. The model was trained using the training data, with 100 observations used for validation (k=5). A threshold of 0.5 was used to produce the confusion matrix from prediction values. Classification error rates were also calculated, along with an F1 score.

```
# Creating validation set from training data
set.seed(1)
val_index <- sample(1:nrow(diabetes_training), 100) #randomly select 100 obsv for val

validation_data <- diabetes_training[val_index, ]
training_data_knn <- diabetes_training[-val_index, ]

# Splitting X and Y (variables and type_f)
vars <- c("npreg", "age", "glu", "bmi", "bp", "skin", "ped")

train_x <- training_data_knn[, vars]
train_y <- training_data_knn$type_f
```

```r
val_x <- validation_data[, vars]
val_y <- validation_data$type_f

test_x <- diabetes_test[, vars]
test_y <- diabetes_test$type_f

# Scaling using training only
train_scaled <- scale(train_x)

val_scaled <- scale(val_x,
                    center = attr(train_scaled, "scaled:center"),
                    scale = attr(train_scaled, "scaled:scale"))
test_scaled <- scale(test_x,
                     center = attr(train_scaled, "scaled:center"),
                     scale = attr(train_scaled, "scaled:scale"))

# Choosing best k value
k_values <- 1:20
val_errors <- rep(NA, length(k_values))

for (k in k_values) {
  pred_knn <- knn(train_scaled, val_scaled, cl = train_y, k = k)
  cm_knn <- table(pred_knn, val_y)

  error_knn <- (cm_knn["Y","N"] + cm_knn["N","Y"]) / sum(cm_knn)

  val_errors[k] <- error_knn
}

best_k <- which.min(val_errors)

# Final model on test data
pred_knn_final <- knn(train_scaled, test_scaled, cl = train_y, k = best_k)

cm_knn_final <- table(Predicted = pred_knn_final, Actual = test_y)
cm_knn_final
```

```
##          Actual
## Predicted   N   Y
##         N 166  60
##         Y  15  41
```

Gradient boosting was also applied to the data using a Bernoulli distribution for binary classifications, trained with 5000 trees, interaction depth of 4, and a shrinkage parameter of 0.1. Shrinkage value of 0.1 was selected to provide a balance between model flexibility and overfitting, allowing gradual learning while maintaining stable performance. A threshold of 0.5 was used to produce the confusion matrix from prediction values. Classification error rates were also calculated, along with an F1 score.

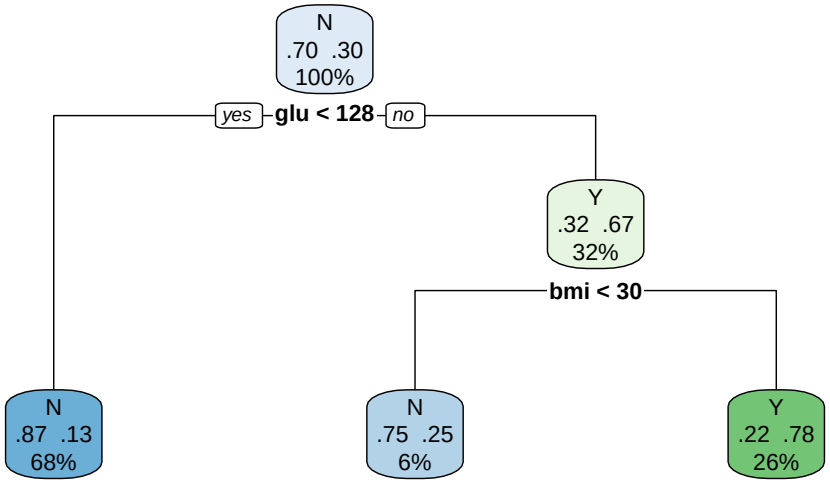
```r
# Convert Y and N to 1 and 0
train_y_boost <- ifelse(diabetes_training$type_f == "Y", 1, 0)
test_y_boost <- ifelse(diabetes_test$type_f == "Y", 1, 0)

# Fitting boosting model
```

```r
boost_model <- gbm(
  train_y_boost ~ npreg + age + glu + bmi + bp + skin + ped,
  data = diabetes_training,
  distribution = "bernoulli",
  n.trees = 5000,
  interaction.depth = 4,
  shrinkage = 0.1
)

# Predicting  probabilities
probs_boost <- predict(boost_model,
                       diabetes_test,
                       n.trees = 5000,
                       type = "response")

# Converting probabilities back into Y and N classes
pred_boost <- ifelse(probs_boost > 0.5, "Y", "N")

# Creating confusion matrix
cm_boost <- table(Predicted = pred_boost,
                  Actual = diabetes_test$type_f)

cm_boost
```

```
##          Actual
## Predicted   N   Y
##         N 152  52
##         Y  29  49
```

A pruned tree model was applied to the data. A threshold 0.5 was used for classification from prediction values. Performance metrics and an F1 score was produced. The tree plot and confusion matrix are as seen below:

```r
# Fitting full tree
tree_model <- rpart(
  type_f ~ npreg + age + glu + bmi + bp + skin + ped,
  data = diabetes_training,
  method = "class"
)

# Find optimal CP
best_cp <- tree_model$cptable[
  which.min(tree_model$cptable[,"xerror"]),
  "CP"
]

# Pruning the tree
pruned_tree <- prune(tree_model, cp = best_cp)

# Plot the tree
rpart.plot(pruned_tree, type = 2, extra = 104)
```



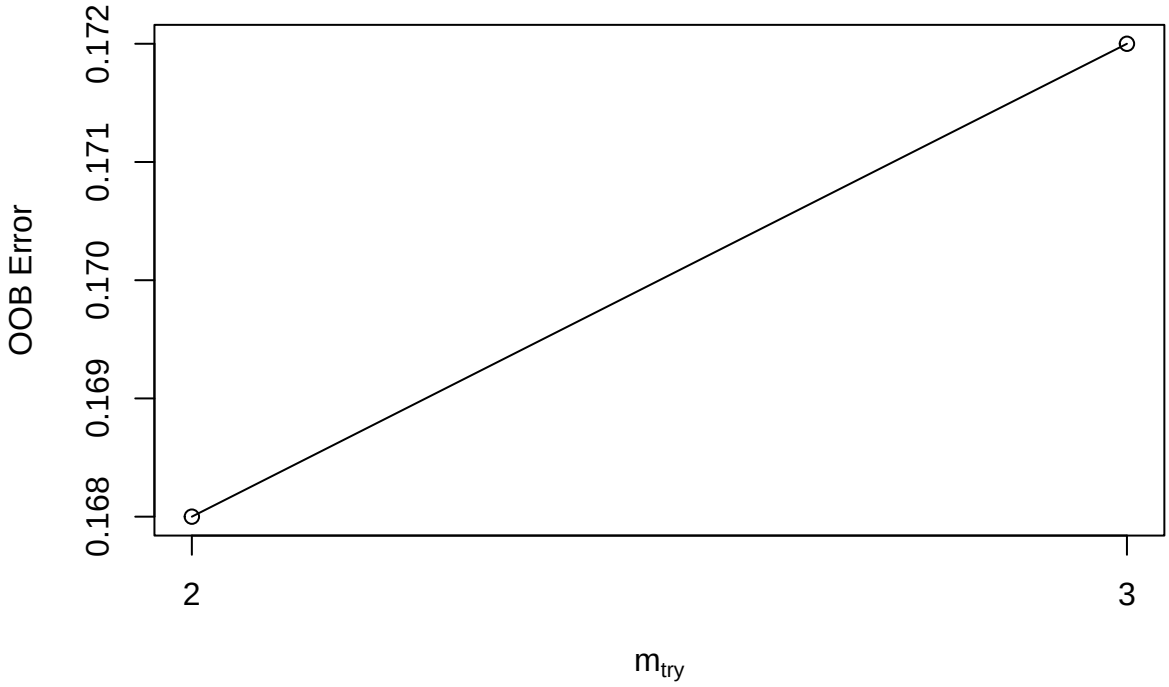

```
# Predictions on test data
tree_pred <- predict(pruned_tree,
                     diabetes_test,
                     type = "class")

# Confusion matrix
cm_tree <- table(Predicted = tree_pred,
                 Actual = diabetes_test$type_f)

cm_tree
```

```
##           Actual
## Predicted   N   Y
##         N 151  44
##         Y  30  57
```

Random forest model with 500 trees was applied to the data. A threshold 0.5 was used for classification from prediction values. Performance metrics and an F1 score was produced.

```
# Fitting initial model
rf_model <- randomForest(
  type_f ~ npreg + age + glu + bmi + bp + skin + ped,
  data = diabetes_training,
  ntree = 500,
  importance = TRUE
)

# Tuning mtry as default may not be optional
set.seed(1)

mtrytune <- tuneRF(
  diabetes_training[, c("npreg", "age", "glu", "bmi", "bp", "skin", "ped")],
  diabetes_training$type_f,
  stepFactor = 1.5,
  improve = 0.01,
```

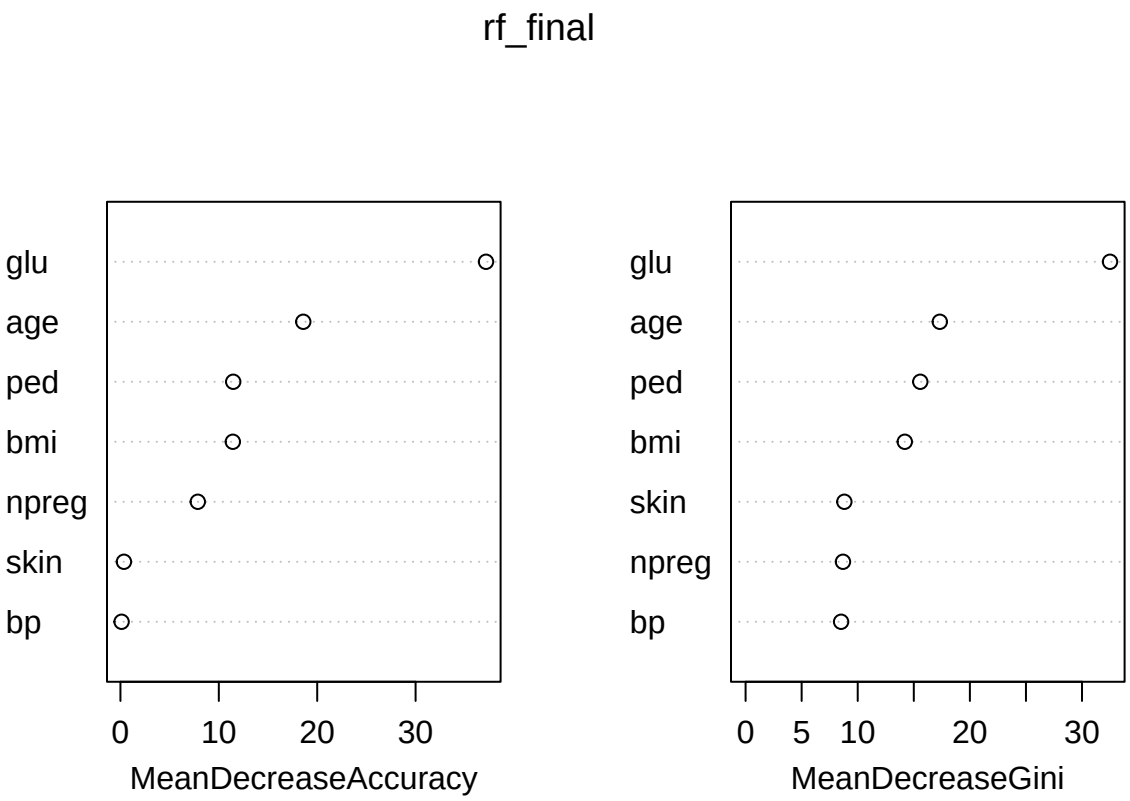
```
  ntreeTry = 500
)
```




```
# Fit final tuned model
best_mtry <- mtrytune[which.min(mtrytune[,2]), 1]

rf_final <- randomForest(
  type_f ~ npreg + age + glu + bmi + bp + skin + ped,
  data = diabetes_training,
  ntree = 500,
  mtry = best_mtry,
  importance = TRUE
)

# Predictions on test data
rf_pred <- predict(rf_final, diabetes_test)

# Confusion matrix
cm_rf <- table(Predicted = rf_pred,
               Actual = diabetes_test$type_f)

cm_rf
```

```
##           Actual
## Predicted   N   Y
```

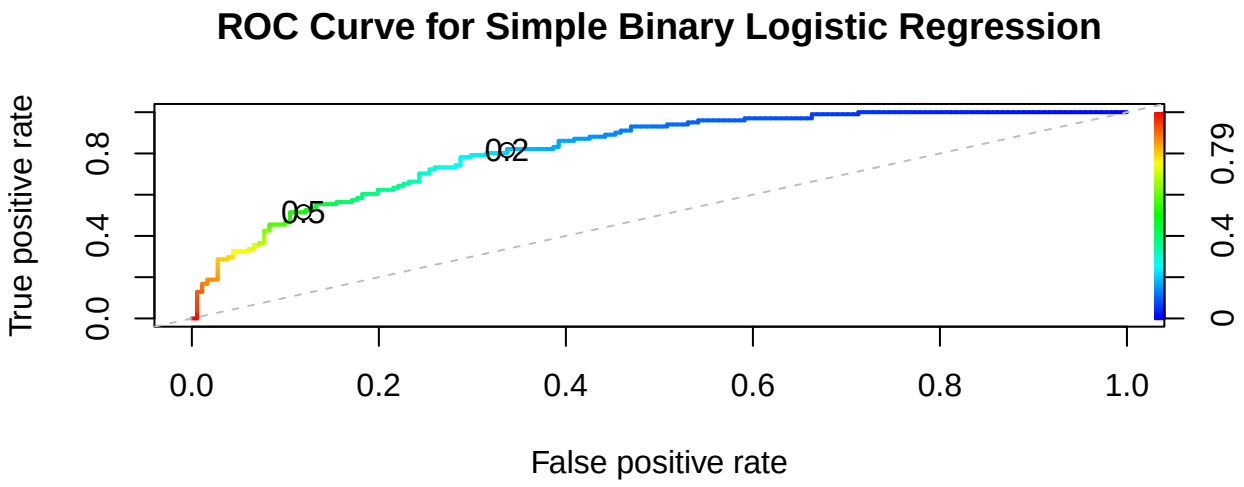
```
##          N 154  48
##          Y  27  53
```



ROC curves and Area Under Curve (AUC) measures were produced for simple binary logistic regression and gradient boosting models as shown below. The code for the boosting model graph has been omitted due to similarity.

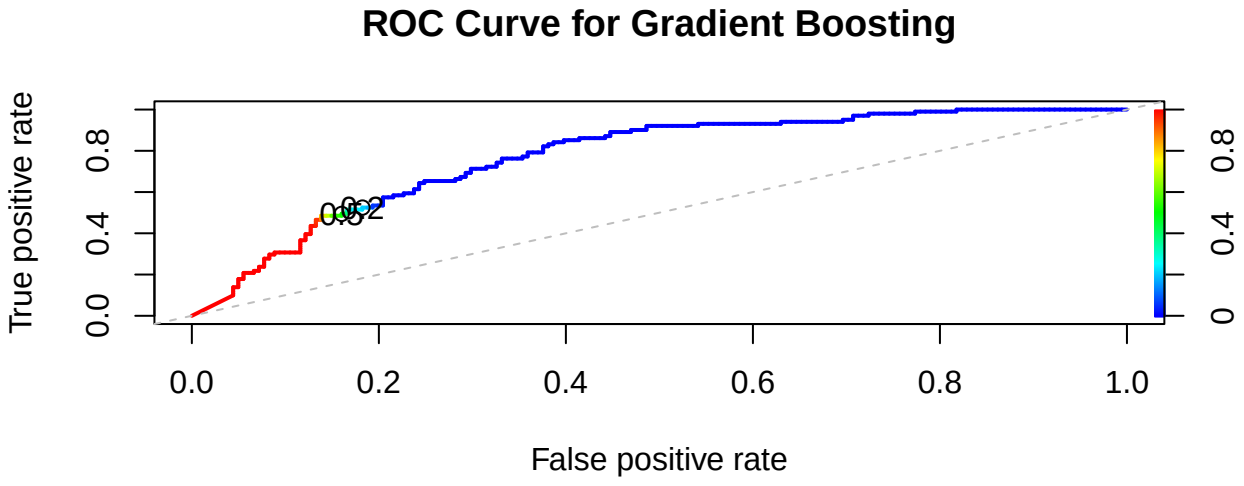
```r
# ROC curve for simple binary logistic regression model
# Creating prediction object
pred_simple_obj <- prediction(
  predictions = probs_simple,
  labels = diabetes_test$type_f
)

# Computing ROC curve
roc_simple <- performance(
  pred_simple_obj,
  measure = "tpr",
  x.measure = "fpr"
)

# Plotting ROC curve
plot(
  roc_simple,
  colorkey = TRUE,
  colorize = TRUE,
  lwd = 2,
```

```
  main = "ROC Curve for Simple Binary Logistic Regression",
  print.cutoffs.at = c(0.2, 0.5),
  cex.axis = 0.9,
  cex.lab = 1
)

abline(a = 0, b = 1, lty = 2, col = "grey")
```




```
# Computing AUC
auc_simple <- performance(pred_simple_obj, measure = "auc")
auc_simple_value <- auc_simple@y.values[[1]]
auc_simple_value
```

```
## [1] 0.8194847
```



```
## [1] 0.7733166
```

Table 1: Performance metrics for machine learning models

Model	Error	Accuracy	Sensitivity	Specificity	Precision	F1
Logistic Regression	0.2482	0.7518	0.5149	0.8840	0.7123	0.5977
KNN	0.2600	0.7340	0.4059	0.9171	0.7321	0.5223
Boosting	0.2872	0.7128	0.4851	0.8398	0.6282	0.5475
Decision Tree	0.2624	0.7376	0.5644	0.8343	0.6552	0.6064
Random Forest	0.2660	0.7340	0.5248	0.8508	0.6625	0.5856

Task 2.9

Multiple machine learning methods were applied to the diabetes data, and compared using classification error, accuracy, sensitivity, specificity, precision and F1 score. Overall, model performance was broadly similar but there were important differences in their ability to correctly identify diabetes cases.

The simple binary logistic regression model achieved the highest accuracy (75.18%), the lowest error rate (24.82%), and the highest AUC value of 0.8195. This indicates that the model was effective at distinguishing between diabetic and non-diabetic individuals. The model also achieved a relatively high precision of 71.23%, meaning that a large proportion of the predicted diabetic individuals were correctly classified. This model was also easily interpretable, with plasma glucose concentration, body mass index, age, and diabetes pedigree function being identified as significant predictors of diabetes.

The decision tree model achieved the highest sensitivity of 56.44% and the highest F1 score of 0.6064, meaning that it identified the largest proportion of true diabetic cases while maintaining a balance between false negatives and false positives. Precision (65.52%) was slightly worse than logistic regression and K-nearest neaighbours, meaning that there were a greater number of false positive classifications. However, the tree structure was easily interpretable, with glucose concentration and BMI being deemed the most important decision rules.

KNN achieved the highest specificity of 91.71% as well as the highest precision at 73.21%, indicating that when the model predicted diabetes, it was most likely to be correct. However, the model also had the lowest sensitivity, achieving only 40.59%, meaning that many true diabetes cases were missed.

Random forest showed balanced performance in comparison, achieving moderate sensitivity, specificity, and precision values. However, it did not outperform the simpler binary logistic regression model overall. The variable importance measures from the random forest model also showed that glucose concentration was the strongest predictor diabetes, consistent with other models.

Gradient boosting, in this scenario, did not perform as well as the other models. Although boosting is a highly flexible machine learning technique, it produced the lowest accuracy, lowest precision, and highest error, as well as the second lowest sensitivity, specificity, and F1 score. This may be due to the structure of the diabetes data being relatively simple, and better interpreted by simpler models like logistic regression and decision tree.

In conclusion, the simple binary logistic regression model would be best suited for this application. It achieved the strongest performance across various metrics, while remaining highly interpretable. This is an important consideration in a medical setting, as clinicians must understand how predictions are being made. The decision tree model could be considered to be a strong alternative, due to its similar performance in metrics, as well as the easy-to-understand graphical illustration.